

Universidade Federal de Goiás
Instituto de Informática
Prof. Ronaldo Martins da Costa





História

- Em 1980, **Guido van Rossum** começou a trabalhar em uma nova linguagem de programação. Sua motivação era criar uma linguagem fácil de ler e escrever, com uma sintaxe clara e intuitiva
- **Rossum** trabalhou na Centrum Wiskunde & Informatica (CWI), um centro de pesquisa na Holanda. Fazia parte de um projeto de desenvolvimento de um sistema de interpretação de linguagem de programação chamado ABC
- O ABC foi projetado para ser uma linguagem simples e didática, mas, apesar de suas boas intenções, não ganhou ampla adoção



História

- Em fevereiro de 1991, foi lançada a primeira versão pública do Python, marcando oficialmente o nascimento dessa linguagem
- A linguagem de programação Python hoje é uma das mais populares e amplamente utilizadas
- Uma das razões para a popularidade de Python é a sua facilidade de aprendizado e uso



História

- Foi inspirado Ele se inspirou em diversas linguagens de programação, incluindo:
 - Modula-3: Pela sua abordagem modular e pela ênfase na facilidade de desenvolvimento de software
 - C: Pela eficiência e flexibilidade
 - ABC: Pela sintaxe simples e legível
 - Unix: Pela filosofia de ferramentas pequenas e reutilizáveis



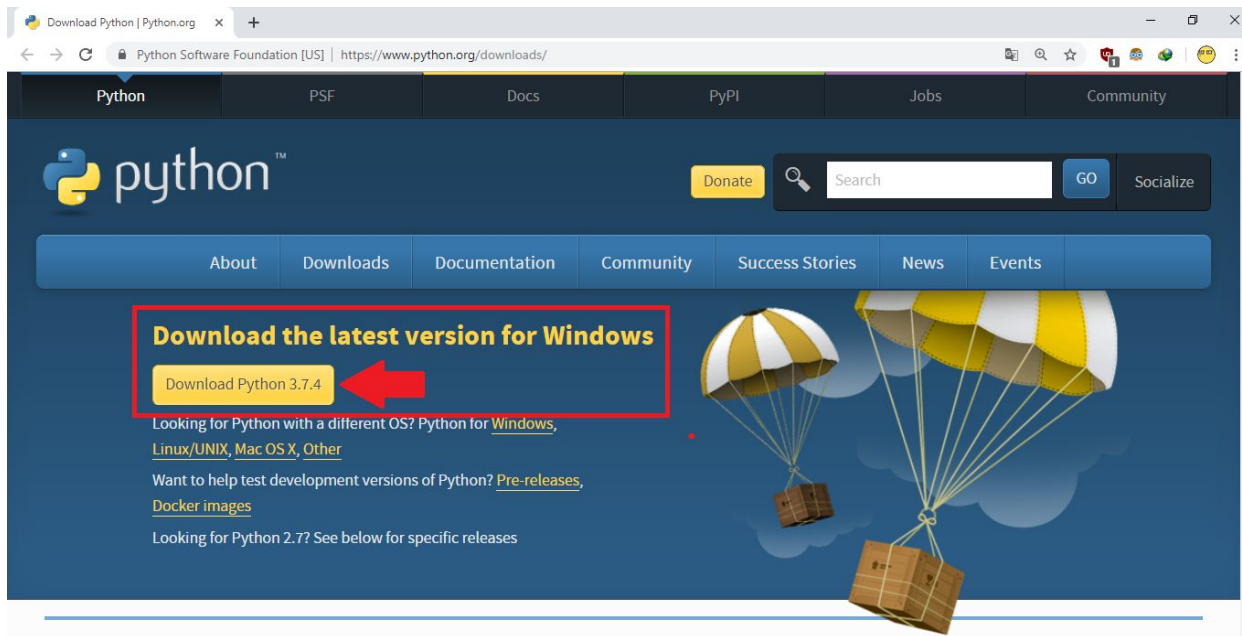
História

- A partir de 2010, o Python começou a se consolidar como uma das linguagens mais populares do mundo, especialmente em áreas como desenvolvimento web, análise de dados, inteligência artificial e automação
- Frameworks como Django e Flask ajudaram a solidificar a posição de Python no desenvolvimento de aplicações web
- Bibliotecas como NumPy, Pandas, TensorFlow e Scikit-learn se tornaram ferramentas essenciais para cientistas de dados e pesquisadores em IA



Instalando o Python

- <http://python.org.br/instalacao-windows/>
- <https://python.org.br/instalacao-linux/>
- <https://python.org.br/instalacao-mac/>





IMPORTANTE: IDENTIFICAÇÃO

- Algunas linguagens utilizam {} para identificar blocos de comandos
- O Python utiliza posição das colunas para definir blocos

```
def nome-da-funcao(parametros):  
    for a in range(1, 100):  
        if (a/10 == 5):  
            print("Valor de 50")  
        else:  
            print("Valor diferente de 50")  
    print("Abaixo do IF")  
print("Fora do FOR")  
return
```

```
function register()  
{  
    if (isEmpty($POST)) {  
        $msg = '';  
        if ($POST['user_name']) {  
            if ($POST['user_password_new']) {  
                if ($POST['user_password_new'] == $POST['user_password_repeat']) {  
                    if (strlen($POST['user_password_new']) > 5) {  
                        if (strlen($POST['user_name']) < 65 && strlen($POST['user_name']) > 1) {  
                            if (preg_match('/^[a-z\d](2,64)$/', $POST['user_name'])) {  
                                $user = read_user($POST['user_name']);  
                                if (!isset($user['user_name'])) {  
                                    if ($POST['user_email']) {  
                                        if (strlen($POST['user_email']) < 65) {  
                                            if (filter_var($POST['user_email'], FILTER_VALIDATE_EMAIL)) {  
                                                create_user();  
                                                $SESSION['msg'] = 'You are now registered so please login';  
                                                header('Location: ' . $_SERVER['PHP_SELF']);  
                                                exit();  
                                            } else $msg = 'You must provide a valid email address';  
                                        } else $msg = 'Email must be less than 64 characters';  
                                    } else $msg = 'Email cannot be empty';  
                                } else $msg = 'Username already exists';  
                            } else $msg = 'Username must be only a-z, A-Z, 0-9';  
                        } else $msg = 'Username must be between 2 and 64 characters';  
                    } else $msg = 'Password must be at least 6 characters';  
                } else $msg = 'Passwords do not match';  
            } else $msg = 'Empty Password';  
        } else $msg = 'Empty Username';  
        $SESSION['msg'] = $msg;  
    }  
    return register_form();  
}
```



Tipos de Variáveis

- Assim como nas outras linguagens, o python possui diferentes “tipo” de variáveis para armazenamento de dados, são eles:
 - Numéricas
 - Ponto Flutuante
 - Textos
 - Listas
 - Tuplas
 - Dicionários



Tipos de Variáveis-Numéricas

- Utilizadas para armazenar valores numéricos. A declaração de qualquer variável em python é feita escrevendo o nome da variável e atribuindo um valor/tipo para a mesma
- Diferente de outras linguagens que possuem uma faixa de valores aceitos para uma variável inteira o python utiliza alocação dinâmica.
- Números pequenos são armazenados diretamente em memória de forma otimizada, números grandes são armazenados em uma estrutura que cresce dinamicamente, consumindo mais memória conforme o número aumenta

```
fatorial = 1
for i in range(1, 5+1):
    fatorial *= i
print(fatorial)
```



Tipos de Variáveis-Ponto Flutuante

- Utilizadas para armazenar valores numéricos com casas decimais
- Pode armazenar até 17 casas decimais

```
import math # Importa a constante pi da biblioteca math
# Definindo o valor do raio
raio = 7
# Calculando a circunferência
circunferencia = 2 * math.pi * raio
# Exibindo o resultado
print(f"A circunferência do círculo com raio {raio} e pi={math.pi} é {circunferencia:.2f}")
pi = 3.14
circunferencia = 2 * pi * raio
# Exibindo o resultado
print(f"A circunferência do círculo com raio {raio} e pi={pi} é {circunferencia:.2f}")
```



Tipos de Variáveis-Texto

- Utilizadas para armazenar valores alfanuméricos (textos)
- O tamanho máximo de uma string (texto) é definido pela quantidade de memória disponível, em função do sistema de alocação dinâmica de memória que o python utiliza

```
s = "Framework de Desenvolvimento Web para consumo de Modelos Treinados de  
Inteligência Artificial"  
print(len(s))  
print(s.upper())  
print(s.lower())  
print(s.capitalize())  
print(s.find("Web"))  
print(s.find("python"))  
print(s.count("hello"))  
print(s.split(' '))  
print(s.isalpha())
```



Tipos de Variáveis-Listas

- O tamanho de uma lista é dinâmico, pode crescer ou diminuir durante a execução do programa.
- Uma lista pode armazenar diferentes tipos de dados dentro da mesma lista, ou seja, ela é heterogênea.

```
num = [1, 2, 3, 4, 5]
```

```
for i in num:
```

```
    print(i)
```

```
num.append(7)
```

```
for i in num:
```

```
    print(i)
```

```
lista = [10, "Texto", 3.14, True]
```

```
for i in lista:
```

```
    print(i)
```



Tipos de Variáveis-Tuplas

- Ao contrário de uma lista, não é possível adicionar ou remover elementos de tuplas
- Uma lista pode armazenar diferentes tipos de dados dentro da mesma lista, ou seja, ela é heterogênea

```
num = (1, 2, 3, 4, 5)
```

```
for i in num:
```

```
    print(i)
```

```
num.append(7)
```

```
for i in num:
```

```
    print(i)
```

```
lista = (10, "Texto", 3.14, True)
```

```
for i in lista:
```

```
    print(i)
```



Estruturas de Controle

● Comando condicional IF

```
# Solicita ao usuário para digitar um número
numero = float(input("Digite um número: "))

# Verifica se o número é positivo, negativo ou zero
if numero > 0:
    print("O número é positivo.")
elif numero < 0:
    print("O número é negativo.")
else:
    print("O número é zero.")

print("Positivo" if numero > 0 else "Negativo")
```



Estruturas de Controle

- Comando condicional **for** já foi visto
- Comando condicional **while**

```
contador = 1

while contador <= 5:
    print(contador)
    contador += 1
```



Orientação a Objetos - Classes

- Classes são estruturas que permitem agrupar dados e comportamentos (funções) em uma única entidade, o que facilita a criação de objetos com características e ações definidas.

```
class Pessoa:
    # Construtor (método __init__) para inicializar os atributos
    def __init__(self, nome, profissao, idade):
        self.nome = nome
        self.profissao = profissao
        self.idade = idade

    # Método para exibir informações da Pessoa
    def exibir_informacoes(self):
        print(f"Pessoa: {self.nome} {self.profissao} ({self.idade})")
```




Orientação a Objetos - Classes

- A criação de uma classe envolve a definição de um construtor (o método especial `__init__`) e a definição de métodos (funções associadas à classe). Uma instância de uma classe é um objeto criado a partir dela, com dados específicos

```
# Método para atualizar a idade da pessoa
def atualizar_idade(self, nova_idade):
    self.idade = nova_idade
    print(f"A idade da pessoa foi atualizada para {self.idade}")

# Criando uma instância da classe Pessoa
cliente = Pessoa("Ronaldo", "Professor", 33)

# Chamando o método da classe
cliente.exibir_informacoes() # Exibe as informações do carro

# Atualizando a idade da pessoa
cliente.atualizar_idade(12)

# Exibindo as informações novamente após a atualização
cliente.exibir_informacoes()
```



Orientação a Objetos-Herança

- Herança é um mecanismo que permite que uma classe herde atributos e métodos de outra classe, promovendo o reuso de código e a extensão de funcionalidades sem a necessidade de reescrever o código da classe base

```
# Classe Pai
class Pessoa:
    def __init__(self, nome, profissao, idade):
        self.nome = nome
        self.profissao = profissao
        self.idade = idade

    def exibir_informacoes(self):
        print(f"Pessoa: {self.nome}, {self.profissao} ({self.idade} anos)")

    def atualizar_idade(self, nova_idade):
        self.idade = nova_idade
        print(f"A idade de {self.nome} foi atualizada para {self.idade} anos.")
```



Orientação a Objetos-Herança

● Continuação...

```
# Classe Filha
class Funcionario(Pessoa):
    def __init__(self, nome, profissao, idade, salario):
        # Chamando o construtor da classe Pai (Pessoa)
        super().__init__(nome, profissao, idade)
        self.salario = salario

    def exibir_informacoes(self):
        # Sobrescrevendo o método da classe Pai
        super().exibir_informacoes() # Chama o método exibir_informacoes da
        classe Pessoa
        print(f"Salário: R${self.salario:.2f}")

    def aumentar_salario(self, aumento):
        self.salario += aumento
        print(f"O salário de {self.nome} foi aumentado para
        R${self.salario:.2f}")
```



Orientação a Objetos-Herança

● Continuação...

```
# Criando uma instância da classe Funcionario (classe filha)
funcionario = Funcionario("Carlos", "Engenheiro", 40, 5000)

# Chamando o método da classe Funcionario
funcionario.exibir_informacoes() # Exibe informações do funcionário (nome,
    profissão, idade e salário)

# Atualizando a idade do funcionário
funcionario.atualizar_idade(41)

# Aumentando o salário do funcionário
funcionario.aumentar_salario(1000)

# Exibindo as informações novamente após a atualização
funcionario.exibir_informacoes()
```



Orientação a Objetos-Encapsulamento

- Visa restringir o acesso direto a determinados atributos e métodos de uma classe, fornecendo uma interface controlada para interagir com eles. Isso ajuda a proteger os dados da classe, controlando como eles são acessados e modificados

```
class Pessoa:
    def __init__(self, nome, profissao, idade):
        self.nome = nome
        self.profissao = profissao
        self.__idade = idade # Atributo privado (encapsulado)

# Getter: Método para acessar o atributo privado 'idade'
def get_idade(self):
    return self.__idade
```



Orientação a Objetos-Encapsulamento

● Continuação...

```
# Setter: Método para modificar o atributo privado 'idade'
def set_idade(self, nova_idade):
    if nova_idade >= 0:
        self.__idade = nova_idade
        print(f"A idade de {self.nome} foi atualizada para {self.__idade}
anos.")
    else:
        print("Idade inválida! A idade não pode ser negativa.")

# Método para exibir informações da Pessoa
def exibir_informacoes(self):
    print(f"Pessoa: {self.nome}, {self.profissao} ({self.get_idade()}
anos)")

# Criando uma instância da classe Pessoa
cliente = Pessoa("Ronaldo", "Professor", 33)
```



Importando Bibliotecas

- Bibliotecas são coleções de módulos e funções prontas para uso, que facilitam o desenvolvimento e permitem a reutilização de código

```
# Importando a biblioteca math
import math
```

```
numero = 16
raiz_quadrada = math.sqrt(numero) # Calculando a raiz quadrada de 16
print(f"A raiz quadrada de {numero} é: {raiz_quadrada}")
```

```
pi = math.pi # Calculando o valor de pi
print(f"O valor de pi é: {pi}")
```

```
fatorial = math.factorial(5) # Calculando o fatorial de 5
print(f"O fatorial de 5 é: {fatorial}")
```

```
seno = math.sin(math.radians(45)) # Calculando o seno de 45 graus (convertendo
    graus para radianos)
print(f"O seno de 45 graus é: {seno}")
```



Importando Bibliotecas

```
# Importando as funções da biblioteca
import import2b as retangulo
# Função principal do programa
def main():
    # Solicitando os lados do retângulo
    lado1 = float(input("Digite o comprimento do primeiro lado do retângulo:
"))
    lado2 = float(input("Digite o comprimento do segundo lado do retângulo: "))

    # Calculando o perímetro e a área usando as funções da biblioteca retangulo
    perimetro = retangulo.calcular_perimetro(lado1, lado2)
    area = retangulo.calcular_area(lado1, lado2)

    # Exibindo os resultados
    print(f"O perímetro do retângulo é: {perimetro}")
    print(f"A área do retângulo é: {area}")

# Chamando a função principal
if __name__ == "__main__":
    main()
```




Importando Bibliotecas

```
def calcular_perimetro(lado1, lado2):  
    """Calcula o perímetro de um retângulo."""  
    return 2 * (lado1 + lado2)  
  
def calcular_area(lado1, lado2):  
    """Calcula a área de um retângulo."""  
    return lado1 * lado2
```

Universidade Federal de Goiás
Instituto de Informática
Prof. Ronaldo Martins da Costa

